

TfL What-If Simulator

Technical Architecture and Design Decisions

Document Structure

- 1. Architecture overview**
- 2. Frontend component architecture**
- 3. Data and API architecture**
- 4. Routing and simulation model**
- 5. Live train movement architecture**
- 6. State management decisions**
- 7. Non-functional considerations**
- 8. Risks and future work**

1. Architecture Overview

The system follows a layered architecture. The user interacts with a Next.js client interface. The frontend requests graph data from internal API routes. Those API routes query Neo4j or TfL services and return structures suitable for the map. The map then renders stations, edges, routes, trains, overlays and control panels. What-If simulation state is kept client-side and is applied to routing as constraints rather than being written back to the graph database.

Layer	Responsibility	Project examples
Presentation layer	Map, sidebars, floating controls, route panel, search UI and mobile controls.	MapCanvas, LeafletMap, layout components, map layers.
Client state layer	Selected route, search query, closure sets, train filters, HUD state and saved scenarios.	React hooks under app/hooks.
Application logic layer	Graph building, routing, train interpolation and scenario persistence.	graph.js, pathfinding.js, train utility modules, useWhatIfClosures.js.
Internal API layer	Provide station graph and live train feed to frontend.	/api/stations and /api/trains/live.
External/data layer	Neo4j Aura station graph and TfL live APIs.	Neo4j driver and TfL Unified API endpoints.

2. Frontend Component Architecture

MapCanvas acts as the main orchestration component. It does not draw every marker itself; instead, it coordinates state and passes props to focused child components. This is an important design decision because the map screen contains several independent concerns: data loading, route selection, search, live trains, What-If state, sidebars, mobile layout and reset behaviour.

Component	Role	Why it matters
MapCanvas.jsx	Top-level map container and state coordinator.	Keeps global UI state in one place and wires hooks into visible components.
LeafletMap.jsx	Leaflet map rendering and route interaction.	Contains station/edge loading, route calculation and map-layer rendering.
MapSearchBox.jsx	Station search and focused station actions.	Provides quick station navigation and route start/destination actions.
MapControlPanelContent.jsx	Line controls, train controls, What-If scenario controls and status display.	Centralises control-panel functionality used by sidebar/mobile UI.
MapSidebar.jsx	Desktop control container.	Separates panel shell from actual controls.
MobileControlSheet.jsx	Mobile control container.	Allows controls to be reused on mobile without duplicating business logic.
MapWhatIfOverlay.jsx	Visual What-If guidance/overlay.	Makes the current simulation context clear to the user.
RouteInfoPanel.jsx	Route summary and legs.	Turns route algorithm output into understandable user information.

3. Data and API Architecture

The /api/stations endpoint is the main bridge between the graph database and the frontend map. It queries Neo4j for station-to-station relationships and reshapes them into a JSON structure of nodes and edges. This

avoids exposing database-specific records to the map layer and lets the frontend operate on a clean graph model.

A cache is used for the station graph because the station network does not need to be reloaded on every map interaction. This reduces repeated database work and improves perceived performance. The route also tracks an in-flight request so multiple near-simultaneous requests can share the same loading process.

Endpoint/module	Input	Output	Purpose
/api/stations	No user input. Uses Neo4j connection details from environment.	nodes and edges JSON.	Provides map and route graph data.
/api/trains/live	Optional lineIds query parameter.	TfL arrivals, trackable arrivals, untrackable arrivals and metadata.	Provides live train data for frontend train movement.
useLineStatus	TfL line status endpoint.	Effective open/closed/partial line sets.	Feeds live service state into UI and routing context.
useLineDelays	TfL line status endpoint.	Severity, description, reason and additional information.	Feeds line delay indicators into control panel.
useLiveStationClosures	TfL station disruptions endpoint.	Set of closed station IDs.	Lets normal routing avoid live closed stations.

4. Routing and Simulation Model

Routing is based on a weighted graph. Each station is a node and each Underground connection is an edge with a travel-time weight. The pathfinding function extends basic Dijkstra behaviour by tracking the current line in the state key. This makes it possible to add a penalty when changing from one line to another. As a result, the chosen route is not only short by travel time but also less likely to include unnecessary line changes.

The algorithm accepts three disruption constraints: closed stations, closed lines and blocked edges. Closed stations are skipped unless they are the destination. Closed lines prevent travel along all edges belonging to that line. Blocked edges allow partial closures to remove specific segments without necessarily removing the whole line.

Routing input	How it affects routing
closedStations	Intermediate stations in the set are avoided.
closedLines	Edges whose line ID is in the set are ignored.
blockedEdges	Specific undirected line-edge keys are ignored.
CHANGE_PENALTY_SECONDS	Adds five minutes when changing between lines.
statePath	Retains station and line state for route-leg grouping.

5. Live Train Movement Architecture

The live train feature is designed to be resilient. TfL arrivals are fetched and grouped into trackable vehicle predictions where possible. The application then converts these into train snapshots and interpolated marker positions. Because public live arrival data can be incomplete, the hook falls back to deterministic simulated movement across graph edges. This means the product continues to demonstrate train movement even when the live feed is unavailable or insufficient.

Module	Responsibility
liveTrainFeedService.js	Fetches TfL arrivals and separates trackable predictions.
trainSnapshotBuilder.js	Groups arrival predictions into train snapshots.

trainRouteExpansion.js	Adds estimated intermediate stops when TfL predictions skip stations.
trainPositionBuilders.js	Builds live or fallback train marker positions.
useTrainMovements.js	Polls live data, merges snapshots and exposes train markers to the map.
CanvasTrainLayer.jsx / TrainLayer.jsx	Renders train positions visually.

6. State Management Decisions

The project uses React hooks rather than a global state library. This is appropriate because most state is local to the map screen and can be grouped by concern. For example, station search has its own hook, train filters have their own hook, and What-If closures have their own hook. This improves readability and avoids turning MapCanvas into a single unstructured state file.

State group	Stored values	Reason for separation
HUD state	Zoom and centre coordinates.	Purely presentational map feedback.
Station search	Query, matches, focused/highlighted station.	Search behaviour is independent of route algorithm internals.
What-If closures	Closed station set, closed line set, saved scenarios.	Simulation state must be separate from live state.
Train filters	Show trains, filter mode, selected visible lines.	Visual train preferences should not affect station graph data.
Route panel	Route error, route summary, panel open/collapsed state.	Keeps route display separate from route calculation.

7. Non-Functional Considerations

Performance

The station graph is cached server-side and map events are debounced. Vector rendering uses an expanded SVG padding value so paths remain visible while dragging near viewport edges. Train animation uses regular ticks but can be disabled in What-If mode to reduce visual complexity.

Usability

The What-If toggle was moved to the main screen after questionnaire feedback. Search starts collapsed to reduce first-load clutter. Route auto-fit improves route visibility after selecting two stations.

Maintainability

Feature concerns are split into hooks and components. API routes translate external data into internal structures, preventing database/API shape from leaking throughout the UI.

Reliability

Live APIs can fail. The product includes fallback line options and fallback train movement so the map remains usable during API issues.

Data integrity

What-If mode does not write hypothetical closures back to Neo4j. This protects the canonical station graph and keeps simulation reversible.

8. Risks and Future Work

Risk / limitation	Impact	Possible improvement
Browser-local scenarios	Saved scenarios are not available across devices.	Add backend persistence with user accounts.
TfL API dependency	Live data may be unavailable, delayed or incomplete.	Improve fallback messaging and cache recent successful responses.
Complex map UI	Many floating panels can overlap without careful layout.	Introduce a formal responsive layout grid and design tokens.
Route model simplification	Does not include crowding, accessibility or service frequency.	Add richer edge weighting and user route preferences.
Testing coverage	Current tests cover important units but not full end-to-end scenarios.	Add integration/UI tests for route selection and What-If scenarios.